WIKIPEDIA

## Inhaltsverzeichnis


1 Geschichte
2 Das Apache-Logging-Projekt
3 Funktionsweise
  3.1 Ausgabe-Umfang
  3.2 Appender
  3.3 Konfiguration
    3.3.1 Beispiel
4 Apache Log4j 2
  4.1 Log4j 2 Konfigurationsbeispiel
5 Bekanntwerden einer Sicherheitslücke im Dezember 2021
6 Alternativen
7 Literatur
8 Weblinks
9 Einzelnachweise


# Log4j

**Log4j** ist ein Framework für das Logging von Anwendungsmeldungen in Java. Innerhalb vieler Open-Source- und kommerzieller Softwareprodukte hat es sich über die Jahre zu einem De-facto-Standard entwickelt. Log4j gilt als Vorreiter für andere Logging-Frameworks, auch in anderen Programmiersprachen.

**Log4j**

**Basisdaten**

| | |
|---|---|
| **Hauptentwickler** | Scott Deboy, Ralph Goers, Gary Gregory, Christian Grobmeier |
| **Entwickler** | Apache Software Foundation |
| **Aktuelle Version** | 2.26.0[1] (7. Mai 2026) |
| **Betriebssystem** | plattformübergreifend |
| **Programmiersprache** | Java |
| **Lizenz** | Apache-Lizenz 2.0 |

logging.apache.org (https://logging.apache.org/)

## Geschichte

Das Projekt wurde von Ceki Gülcü 1996 während seiner Arbeit am IBM-Entwicklungslabor in Zürich gegründet. Heute ist es ein Teil des Logging-Projekts der Apache Software Foundation und steht unter der Apache-Lizenz 2.0. Es entstand zu einer Zeit, als es in den Java-Standardbibliotheken noch keine Logging-Mechanismen gab. Heutzutage ist es durch seine Ausgereiftheit und Konfigurierbarkeit für viele Softwareentwickler das Log-System der ersten Wahl.

Die Ausstrahlung der Log4j-Konzepte auf andere Programmiersprachen bzw. Plattformen ist so groß, dass es mittlerweile etliche Adaptionen gibt. Einige werden vom Apache Logging Projekt selbst gepflegt. Zum Beispiel:

- log4cxx für C++ mit zu Log4J kompatiblen Konfigurationsdateien
- log4Net für das .Net-Framework
- log4php für PHP

Viele Varianten werden jedoch außerhalb von Apache Logging entwickelt:



Logo von Log4js

- log4C für C
- log4cplus für C++
- log4cpp für C++
- Log4Qt für Qt
- Log4js für JavaScript
- log4perl für Perl
- log4D für Delphi
- log4sh für Unix-Shells
- log4plsql für Oracle PL/SQL
- log4sas für SAS Institute
- AndroidLoggingLog4J[2]
- Modul *logging* für Python[3]
- logger für LabVIEW[4]

Seit Juli 2014 ist Log4j 2 als Nachfolger des Log4j 1.x verfügbar.

## Das Apache-Logging-Projekt

Das Apache-Logging-Projekt versucht, Log4j-ähnliche Systeme für diverse Programmiersprachen zusammenzuführen. Bisher sind das log4j, log4cxx, log4net, log4php[5] und Chainsaw (ein Logdatei-Betrachter und -Analysewerkzeug).[6]

Außerdem werden sogenannte Companions entwickelt, die zusätzliche Funktionalität für Apache Log4j bereitstellen.

## Funktionsweise

Anstatt auftretende Fehler und Infomeldungen auf der Standardausgabe auszugeben, dient Log4j dazu, die Meldungen über sogenannte Logger an das gewählte Ausgabeziel weiterzuleiten („Appender“).[7] Neben der Auswahl des Ausgabeziels (z. B. eine Logdatei) wird gleichzeitig aufgrund der Wichtigkeit („Log-Level“) der Meldung entschieden, ob diese überhaupt weitergeleitet wird. Der Programmierer muss sich beim Erstellen des Programms nur um die Wichtigkeit der Meldungen Gedanken machen. Für eine Meldung können mehrere Appender mit unterschiedlicher Wichtigkeit angeschlossen werden, so dass ein Appender hier die Rolle eines Kanals spielt. Die Filterung und Art der Ausgabe kann zur Laufzeit konfiguriert werden.

Log4j ist auf möglichst hohe Geschwindigkeit ausgelegt, damit das Protokollieren die Systemleistung nicht negativ beeinflusst. So dauert die Entscheidung, ob eine Meldung ausgegeben werden muss, auch auf einem veralteten System (AMD Duron mit 800 MHz, JDK 1.3.1) nur 5 Nanosekunden, die Ausgabe selbst – je nachdem, welches Layout verwendet wird – zwischen 21 und 37 Mikrosekunden.[8]

### Ausgabe-Umfang

In der Konfigurationsdatei kann die Ausgabe je nach *Wichtigkeit* der Nachrichten gefiltert werden. Der Ausgabe-Umfang sinkt mit der zugewiesenen Wichtigkeitsstufe und umfasst alle Nachrichten der Stufe selbst sowie aller noch dringenderen Stufen. Die Reihenfolge stellt sich dabei wie folgt dar: `ALL` → `TRACE` → `DEBUG` → `INFO` → `WARN` → `ERROR` → `FATAL` → `OFF`.

Für die Einstufung der Wichtigkeit gelten folgende Richtwerte:

**`ALL`**
: Alle Meldungen werden ungefiltert ausgegeben

**`TRACE`**
: ausführlicheres Debugging (ab Version 1.2.12[9]), Kommentare

**`DEBUG`**
: allgemeines Debugging (Auffinden von Fehlern)

**`INFO`**
: allgemeine Informationen (Programm gestartet, Programm beendet, Verbindung zu Host `Foo` aufgebaut, Verarbeitung dauerte soundsoviel Sekunden …)

**`WARN`**
: Auftreten einer unerwarteten Situation

**`ERROR`**
: Fehler (Ausnahme wurde abgefangen. Bearbeitung wurde alternativ fortgesetzt)

**`FATAL`**
: Kritischer Fehler, Programmabbruch

**`OFF`**
: Logging ist deaktiviert

### Appender

Mittels Appender kann das Ausgabeziel/können die Ausgabeziele der erzeugten Protokollausgaben festgelegt werden.

Nachfolgend die wichtigsten Appender-Arten:

**`ConsoleAppender`**
: Gibt auf Standardausgabe aus

**`FileAppender`**
: Schreibt in eine Datei

**`RollingFileAppender`**
: Beginnt eine neue Datei ab einer gewissen Größe

**`DailyRollingFileAppender`**
: Beginnt zu gewissen Zeiten mit einer neuen Datei

**`SyslogAppender`**
: Protokolliert mittels Syslog-Dienst

**`JDBCAppender`**
: Schreibt in eine Datenbank

**`NTEventLogAppender`**
: Schreibt ins Ereignisprotokoll des Windows-Systems

**`SMTPAppender`**
: Schickt bei gewissen Meldungen eine Mail.

**`SocketAppender`**
: Sendet die Protokollnachricht an einen konfigurierten Socket-Listener.

**`LogCatAppender`**
: Protokolliert in den Android `LogCat`

Weitere Appender können jederzeit hinzugefügt werden.

## Konfiguration

Es gibt drei Arten, Log4j zu konfigurieren: mittels einer Properties- oder einer XML-Datei oder im Programmcode. Es wird empfohlen, eine Properties- oder XML-Datei zu verwenden, damit ist die Konfiguration vom Code getrennt, was es ermöglicht, ohne Modifikation oder Neustart der Applikation die Konfiguration des Loggings anzupassen. Somit kann beispielsweise eine Applikation so lange nur mit der Wichtigkeitsstufe *FATAL* betrieben werden, bis ein Fehler auftritt. Ab dann wird die Stufe *WARN* gesetzt, ohne die Applikation anzuhalten.

Die Konfigurationsdateien definieren mittels der folgenden Komponenten das Verhalten von Log4j:

**Appender**
: Diese bestimmen mittels Konfiguration der entsprechenden Appender-Klasse, wohin die Protokollausgaben geschrieben werden sollen, und mittels Konfiguration des Layouts, wie dorthin geschrieben werden soll. Neben der reinen Nachricht können mittels Muster zusätzlich Wichtigkeit, Datum, Name des Loggers, Klassenname und Methodenname bis hin zur genauen Codezeile ausgegeben werden.

**Logger**
: Ein Logger ist ein Objekt, welches die Protokollierungsaufgaben einer Klasse übernimmt. Es wird über seinen Namen identifiziert. Üblicherweise wird der Klassenname als Name für den Logger verwendet. Es ist allerdings auch möglich, einen Fantasie-Namen zu verwenden, der von mehreren Klassen benutzt wird. Ein typisches Beispiel könnte so aussehen: `Logger log = Logger.getLogger(org.wikipedia.MyClass)`.
Damit kann das Protokollierungsverhalten einer Klasse gesteuert werden. Da der Logger auch die Paket-Informationen erhält, ist es auch möglich Logging für ganze (Super)-Pakete zu konfigurieren.
Die Konfigurationsdatei kann dann eben den Logger-Namen – also z. B. „org.wikipedia.MyClass“ – und auch die gewünschte Wichtigkeitsstufe (Log-Level) enthalten. Es ist wie bereits erwähnt möglich, eine Konfiguration für ein Paket wie „org.wikipedia“ festzulegen.
Darüber hinaus können entweder allgemein oder auch zu jedem Logger spezifisch ein oder mehrere Appender definiert werden. Somit ist es beispielsweise möglich, fatale Fehler nicht nur in eine Datei zu schreiben, sondern auch gleichzeitig per E-Mail an einen Administrator zu schicken.
Log4j 2.0 bietet neben diesen klassischen Konfigurationen auch modernere Möglichkeiten an, in das Protokollierungsverhalten einzugreifen. Beispielsweise werden sogenannte Marker unterstützt.

Ein weiteres nützliches Merkmal ist der Mapped Diagnostic Context. Dabei wird einer Kontextvariablen ein Wert zugewiesen und in der Konfigurationsdatei kann darauf referenziert werden. Dabei hat jeder Thread seinen eigenen Kontext und kann zusätzliche Informationen wie z. B. die Adresse des Clients bei einer Serveranwendung protokollieren.

### Beispiel

Die folgende XML-Konfiguration konfiguriert eine Applikation so, dass FATAL-Fehler von Fremdbibliotheken auf die Konsole protokolliert werden, ERROR-Fehler der eigenen Applikation zusätzlich auch per E-Mail geschickt werden, bei einer spezifischen Komponente darüber hinaus auch noch INFO-Meldungen protokolliert werden und bei einer bestimmten Klasse sogar noch DEBUG-Meldungen.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE log4j:configuration PUBLIC "-//APACHE//DTD LOG4J 1.2//EN"
    "http://logging.apache.org/log4j/1.2/apidocs/org/apache/log4j/xml/doc-files/log4j.dtd">
<log4j:configuration>
  <!-- Appender für eine einzeilige Ausgabe an der Konsole -->
  <appender name="Konsole" class="org.apache.log4j.ConsoleAppender">
    <layout class="org.apache.log4j.PatternLayout">
      <param name="ConversionPattern"
        value="%d{ABSOLUTE} %5p %c{1}:%L - %m%n" />
    </layout>
  </appender>

  <!-- Appender für dieselbe Ausgabe via E-Mail -->
  <appender name="mail" class="org.apache.log4j.net.SMTPAppender">
    <param name="SMTPHost" value="smtp.myservername.xx" />
    <param name="From" value="email@fromemail.xx" />
    <param name="To" value="toemail@toemail.xx" />
    <param name="Subject" value="[LOG] ..." />
    <param name="BufferSize" value="1" />
    <param name="threshold" value="error" />
    <layout class="org.apache.log4j.PatternLayout">
      <param name="ConversionPattern"
        value="%d{ABSOLUTE} %5p %c{1}:%L - %m%n" />
    </layout>
  </appender>

  <!-- ERROR-Logger für alle Klassen meiner Applikation -->
  <logger name="com.myapp">
    <level value="error"/>
    <appender-ref ref="Konsole" />
    <appender-ref ref="mail" />
  </logger>

  <!-- INFO-Logger für eine spezifische Komponente -->
  <logger name="com.myapp.mycomponent">
    <level value="info"/>
  </logger>

  <!-- DEBUG-Logger für eine spezifische Klasse -->
  <logger name="com.myapp.mycomponent.MyClass">
    <level value="debug"/>
  </logger>

  <!-- FATAL-Logger für die gesamte Applikation (inkl. Bibliotheken) -->
  <root>
    <level value="fatal" />

    <!-- protokolliert auf Konsole - wenn nicht in Sub-Loggern anders definiert -->
    <appender-ref ref="Konsole" />
  </root>
```

```
</log4j:configuration>
```

## Apache Log4j 2

Version 2 wurde von Grund auf neu geschrieben, auch wenn Teile von Log4j 1.x übernommen wurden. Die neue Version verfügt über eine moderne Schnittstelle, wie sie auch von logback bekannt ist. Außerdem unterstützt sie slf4j native. Zugleich wurden Schwächen von logback analysiert und versucht zu verbessern. Somit verliert Log4j 2 beispielsweise keine Logging-Events, wenn sich das System neu konfiguriert. Außerdem wurde eine Plugin-Architektur bereitgestellt und die Konfiguration mittels JSON ermöglicht.[10]

Das Projekt gab im August 2015 bekannt, dass der 1.x-Zweig das „End of Life“ erreicht hat und damit die Weiterentwicklung eingestellt wird. Entwickler werden deswegen gebeten, auf den 2.x-Zweig zu migrieren.[11][12]

Auf der Website von Apache werden einige Unterschiede zwischen Log4j 1 und Log4j 2 aufgelistet.[13]

### Log4j 2 Konfigurationsbeispiel

In Log4j 2 hat sich die Syntax der Konfiguration geändert. Es folgt eine beispielhafte Konfigurationsdatei nach dem neuen Standard:

```
<?xml version="1.0" encoding="UTF-8" ?>
<Configuration >
    <Properties>
        <Property name="logfile">C:/logs/logfile</Property>
        <Property name="LOG_PATTERN">%d{dd.MM.yyyy - HH:mm:ss} %-5p [%t] - %C - %M - %m%n</Property>
    </Properties>

    <Appenders>
        <!-- Appender für eine einzeilige Ausgabe an der Konsole -->
        <Console name="Konsole" target="SYSTEM_OUT">
            <ThresholdFilter level="DEBUG" onMatch="ACCEPT" onMismatch="DENY" />
            <PatternLayout pattern="${LOG_PATTERN}"/>
        </Console>

        <!-- Appender für alle Protokolle ab Stufe Debug in maximal 20 Dateien, die jeweils maximal 20 MB groß werden dürfen -->
        <RollingFile name="DEBUG_LOG"
            fileName="${logfile}_Debug.log"
            filePattern="${logfile}_Debug-%i.log">
            <ThresholdFilter level="DEBUG" onMatch="ACCEPT" onMismatch="DENY" />
            <PatternLayout pattern="${LOG_PATTERN}" />
            <Policies>
                <TimeBasedTriggeringPolicy />
                <SizeBasedTriggeringPolicy size="20000KB" />
            </Policies>
            <DefaultRolloverStrategy max="20" />
        </RollingFile>

        <!-- Appender für alle Protokolle ab Stufe Error in maximal 20 Dateien, die jeweils maximal 20 MB groß werden dürfen -->
        <RollingFile name="ERROR_LOG"
            fileName="${logfile}/${project}_Error.log"
            filePattern="${logfile}/${project}_Error-%i.log">
            <ThresholdFilter level="ERROR" onMatch="ACCEPT" onMismatch="DENY" />
            <PatternLayout pattern="${LOG_PATTERN}" />
            <Policies>
                <TimeBasedTriggeringPolicy />
                <SizeBasedTriggeringPolicy size="20000KB" />
            </Policies>
            <DefaultRolloverStrategy max="20" />
        </RollingFile>
    </Appenders>
    <Loggers>

        <logger name="com.myapp" additivity="false">
            <AppenderRef ref="DEBUG_LOG" />
            <AppenderRef ref="ERROR_LOG" />
            <AppenderRef ref="Konsole" />
        </logger>

        <Root level="all">
            <AppenderRef ref="Konsole" />
         </Root>
    </Loggers>
</Configuration>
```

## Bekanntwerden einer Sicherheitslücke im Dezember 2021

Am 10. Dezember 2021 wurde eine Zero-Day-Lücke in Log4j-Version 2 bekannt (CVE-2021-44228,[14] oft als **Log4Shell** bezeichnet), die Angreifer ausnutzen konnten, um Code auf dem jeweiligen Hostsystem auszuführen (Remote Code Execution) und somit zum Beispiel Rechenleistung von den infizierten Servern zu benutzen, um Krypto-Mining zu betreiben.[15] Die Schwachstelle wurde vom Alibaba-Cloud-Sicherheitsdienst an Apache gemeldet. Aufgrund der weiten Verbreitung von Log4j und der einfach auszunutzenden Schwachstelle bezeichneten Fachleute die Meldung als „Größte Schwachstelle in der Geschichte des modernen Computing“.[16][17]

Zahlreiche Softwareentwickler, Dienstanbieter, Unternehmen und Behörden waren potentiell betroffen, unter anderem Amazon Web Services, Steam und iCloud.[18][19][17] Laut Bundesamt für Sicherheit in der Informationstechnik (BSI) ergaben Berichte von CERT-Quellen, dass weltweite Massenscans und versuchte Kompromittierungen, sowie laut F-Secure auch Ransomware-Angriffe im Zuge der Zero-Day-Lücke stattfanden.[20][19] Das BSI verwies auf eine „unvollständige Auflistung“ von 140 Unternehmen, die besonders anfällig seien, schätze die direkte Bedrohung für Endanwender jedoch als eher gering ein.[21][19][17]

Bekannte Fälle, in denen die Sicherheitslücke kriminell ausgenutzt wurde, sind (Auswahl):[22]

- Am 9. Dezember wurde der erste Angriff auf einen Server des Spiels Minecraft dokumentiert.[17]
- Am 16. Dezember 2021 wurden mit dem Internet verbundene Systeme des belgischen Militärs über die Lücke angegriffen.[23]
- Am 17. Dezember 2021 wurde die Website des Bundesfinanzhofs wegen einer solchen Attacke abgeschaltet.[24]

Die Schließung der Sicherheitslücke war nach kurzer Zeit über eine Softwareaktualisierung möglich, die Umsetzung durch die jeweils betroffenen Softwarehersteller zog sich jedoch aufgrund der Komplexität der Nutzung von Log4j über mehrere Wochen und teilweise sogar Monate hin.[25] Im März 2023, 15 Monate nach Entdeckung und Schließung der Sicherheitslücke, waren immer noch mehr als ein Drittel der täglich heruntergeladenen Versionen von Log4j von der Sicherheitslücke betroffen. Dies wird auf automatische Softwareverteilung und -downloads durch Abhängigkeiten zurückgeführt.[26]

## Alternativen

- Java Logging – seit Java 1.4 Bestandteil der Java-Klassenbibliothek; ähnlich wie Log4j, weniger Appender, kein PatternLayout
- Apache Commons Logging – Schnittstelle für beliebig austauschbare Logging-Frameworks, unter anderem Log4j
- tinylog – schlankes Logging-Framework mit einem statischen Logger[27]
- slf4j mit logback – weit verbreitete moderne Alternative

## Literatur


- Samudra Gupta: *Pro Apache Log4j.* Apress, 22. Juni 2005, ISBN 978-1-59059-499-5, http://www.apress.com/book/view/9781590594995
- Ceki Gülcü: *The Complete Log4j Manual.* QOS.ch, 7. Mai 2003, ISBN 978-2-9700369-0-6, Leseprobe auf Google Books (https://books.google.de/books?id=hZBimlxiyAcC)


## Weblinks

- log4j-Webseiten (https://logging.apache.org/log4j/) (englisch)
  - Manual (https://logging.apache.org/log4j/1.2/manual.html) und Artikel (https://logging.apache.org/log4j/1.2/publications.html) (englisch)
- log4j-Wiki (https://wiki.apache.org/logging-log4j/Log4JProjectPages) (englisch, nur für registrierte Benutzer)
- Log4j Einführung mit Tomcat-Beispielen (https://www.lalil una.de/articles/posts/log4j-tutorial.html) von Sebastian Hennebrueder
- CVE-2021-44228 (https://cve.mitre.org/cgi-bin/cvename.cgi?name=2021-44228)
- *Kommentar zu Log4j: Es funktioniert wie spezifiziert.* (https://www.heise.de/meinung/Kommentar-zu-log4j-Es-funktioniert-wie-spezifiziert-6294476.html) In: *heise.de.* 14. Dezember 2021, abgerufen am 16. Dezember 2021.
- Ruth Fulterer: *Log4j wurde 1997 in der Schweiz entwickelt – der Erfinder erzählt.* (https://www.nzz.ch/technologie/log4j-wurde-1997-in-der-schweiz-entwickelt-der-erfinder-erzaehlt-ld.1660571) In: *Neue Zürcher Zeitung.* 16. Dezember 2021, abgerufen am 16. Dezember 2021 (Bericht über ein Gespräch mit dem Entwickler Ceki Gülcü nach der Entdeckung der Sicherheitslücke).
- *Man muss aufpassen, nicht durchzudrehen.* (https://www.faz.net/aktuell/wirtschaft/unternehmen/log4j-entwickler-man-muss-aufpassen-nicht-durchzudrehen-17685794.html) In: *Frankfurter Allgemeine Zeitung.* 16. Dezember 2021, abgerufen am 16. Dezember 2021 (Interview mit dem Entwickler Christian Grobmeier nach der Entdeckung der Sicherheitslücke, hinter Bezahlschranke).
- *Das wichtigste Hobby der Welt.* (https://www.br.de/mediathek/podcast/wild-wild-web/das-wichtigste-hobby-der-welt/2108090) In: *Bayerischer Rundfunk.* 3. Juli 2025, abgerufen am 6. Juli 2025 (Interview mit dem Entwickler Christian Grobmeier, der mit Selbstzweifeln kämpft und ein neues Hobby findet, das sein Leben ändert. Ein Deep Dive in die Welt der Open Source Maintainer, einer Gruppe freiwilliger Programmierer, die das Internet für uns alle am Laufen hält und dabei immer mehr unter Druck gerät.).

## Einzelnachweise


1. *Release 2.26.0*. (https://github.com/apache/logging-log4j2/releases/tag/rel/2.26.0) 7. Mai 2026 (abgerufen am 7. Mai 2026).
2. Rolf Kulemann: *android-logging-log4j.* (https://code.google.com/p/android-logging-log4j/) Logging with Log4J in Android ¦ providing LogCat appender and configuration facade. Abgerufen am 29. November 2011.
3. *14.5 logging -- Logging facility for Python.* (https://docs.python.org/release/2.5.2/lib/module-logging.html) Abgerufen am 16. März 2020.
4. *Logger Toolkit for LabVIEW – Download – VIPM by JKI.* (https://www.vipm.io/package/field_rnd_services_logger/) Abgerufen am 10. Februar 2022.
5. *Apache log4php – Welcome – Apache log4php.* (https://logging.apache.org/log4php/) Abgerufen am 16. März 2020.
6. *Welcome – Apache Logging Services.* (https://logging.apache.org/index.html) Abgerufen am 16. März 2020.
7. *Log4j 2 Appenders.* (https://logging.apache.org/log4j/2.x/manual/appenders.html) In: *Apache Logging Services Project.* Abgerufen am 2. Januar 2023 (englisch).
8. *Apache log4j 1.2 -.* (https://logging.apache.org/log4j/1.2/index.html) Abgerufen am 16. März 2020 (englisch).
9. *Level (Apache Log4j 1.2.17 API).* (https://logging.apache.org/log4j/1.2/apidocs/org/apache/log4j/Level.html#TRACE) Logging.apache.org, 9. Juni 2012, abgerufen am 24. Juli 2014.
10. *The new log4j 2.0.* (https://grobmeier.solutions/the-new-log4j-2-0-05122012.html) Abgerufen am 16. März 2020.
11. *Log4j – Migrating from Log4j 1.x to 2.x.* (https://logging.apache.org/log4j/2.x/manual/migration.html) Abgerufen am 12. Januar 2022.
12. *Apache log4j 1.2 -.* (https://logging.apache.org/log4j/1.2/) Abgerufen am 12. Januar 2022.
13. *Log4j – Migrating from Log4j 1.x.* (https://logging.apache.org/log4j/2.x/manual/migration.html) Abgerufen am 16. März 2020.
14. Vulnerability Details : CVE-2021-44228 (https://www.cvedetails.com/cve/CVE-2021-44228/), auf cvedetails.com
15. Robin Wille: *Warnstufe Rot: Vor dieser Sicherheitslücke in Server-Software warnt das Bundesamt für Sicherheit in der IT.* (https://www.businessinsider.de/wirtschaft/warnstufe-rot-vor-dieser-sicherheitsluecke-in-server-software-warnt-das-bundesamt-fuer-sicherheit-in-der-it/) 12. Dezember 2021, abgerufen am 27. November 2022 (deutsch).
16. Stephan Finsterbusch, Thiemo Heeg: *Softwarelücke Log4j: „Größte Schwachstelle in der Geschichte des modernen Computing“*. In:

*FAZ.NET*. ISSN 0174-4909 (https://zdb-katalog.de/list.xhtml?t=iss%3D%220174-4909%22&key=cql) (faz.net (https://www.faz.net/aktuell/wirtschaft/unternehmen/log4j-softwareluecke-ist-laut-bsi-aeusserst-ernst-zu-nehmen-17682526.html) [abgerufen am 27. November 2022]).

17. Peter Sieben: *Log4J-Sicherheitslücke: Warum der Fehler unvermeidbar war – ingenieur.de.* (https://www.ingenieur.de/technik/fachbereiche/ittk/log4j-sicherheitsluecke-warum-der-fehler-unvermeidbar-war/) 23. Dezember 2021, abgerufen am 27. November 2022 (deutsch).
18. heise online: *Kritische Zero-Day-Lücke in Log4j gefährdet zahlreiche Server und Apps.* (https://www.heise.de/news/Kritische-Zero-Day-Luecke-in-log4j-gefaehrdet-zahlreiche-Server-und-Apps-6291653.html) Abgerufen am 10. Dezember 2021.
19. Max Hoppenstedt: *Log4-J-Schwachstelle: »Leider machen auch Hacker Überstunden«*. In: *Der Spiegel*. 12. Dezember 2021 (spiegel.de (https://www.spiegel.de/netzwelt/web/log4-j-schwachstelle-ja-leute-die-scheisse-brennt-lichterloh-a-760bd03d-42d2-409c-a8d2-d5b13a9150fd) [abgerufen am 12. Dezember 2021]).
20. *Kritische Schwachstelle in log4j veröffentlicht (CVE-2021-44228).* (https://www.bsi.bund.de/SharedDocs/Cybersicherheitswarnungen/DE/2021/2021-549032-10F2.pdf?__blob=publicationFile&v=3) (PDF) In: *www.bsi.bund.de.* Abgerufen am 14. Dezember 2021.
21. *BlueTeam CheatSheet * Log4Shell* | Last updated: 2021-12-14 0006 UTC.* (https://gist.github.com/SwitHak/b66db3a06c2955a9cb71a8718970c592) Abgerufen am 14. Dezember 2021 (englisch).
22. Christof Kerkmann: *Gefährliche Sicherheitslücke: Die Angriffe auf Unternehmen und Behörden beginnen.* (https://www.handelsblatt.com/technik/it-internet/java-bibliothek-log4j-gefaehrliche-sicherheitsluecke-die-angriffe-auf-unternehmen-und-behoerden-beginnen-/27900826.html) In: *handelsblatt.com.* 17. Dezember 2021, abgerufen am 18. Dezember 2021.
23. *Log4j-Schwachstelle: Belgisches Militär von Angriff über Sicherheitslücke betroffen – Der Spiegel.* (https://www.spiegel.de/netzwelt/web/log4j-schwachstelle-belgisches-militaer-von-angriff-ueber-sicherheitsluecke-betroffen-a-aaf2d48c-84e8-4839-8f0b-77a1c4031fdd) In: *spiegel.de.* 21. Dezember 2021, abgerufen am 23. Dezember 2021.
24. *Schwachstelle Log4j: Bundesfinanzhof schaltet Website nach Hackerangriff ab.* (https://www.faz.net/aktuell/wirtschaft/digitec/log4j-nach-hackerangriff-schaltet-bundesfinanzhof-website-ab-17689399.html) In: *faz.net.* 17. Dezember 2021, abgerufen am 17. Dezember 2021.
25. Alexander Klay: *Log4j: Was über die Sicherheitslücke bekannt ist und wie man sich schützt.* (https://www.morgenpost.de/vermischtes/article234077919/log4j-bsi-zero-day-sicherheitsluecke.html) 13. Dezember 2021, abgerufen am 27. November 2022 (deutsch).
26. heise online: *Log4Shell: Open Source als Gefahr für die Software-Lieferkette.* (https://www.heise.de/meinung/Log4Shell-Open-Source-als-Gefahr-fuer-die-Software-Lieferkette-7606506.html) Abgerufen am 22. März 2023.
27. *tinylog 2 – Lightweight Logging Framework for Java, Scala, Kotlin, and Android.* (https://tinylog.org/v2/) Abgerufen am 31. Dezember 2021.

---

Abgerufen von „https://de.wikipedia.org/w/index.php?title=Log4j&oldid=262460344“